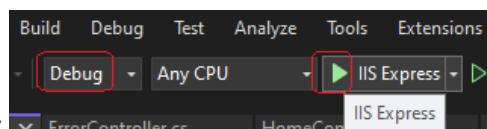
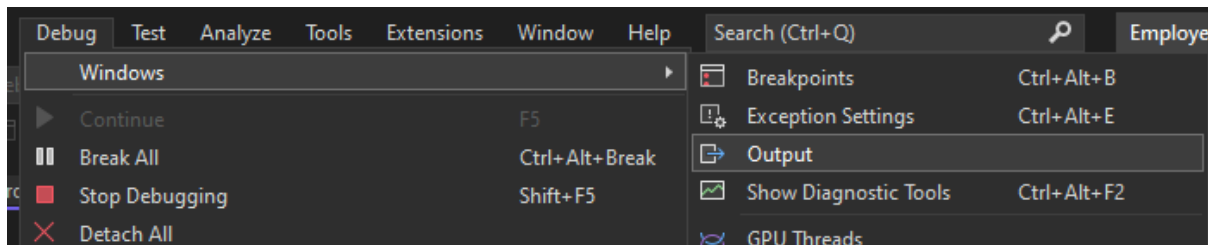


61 – Logging

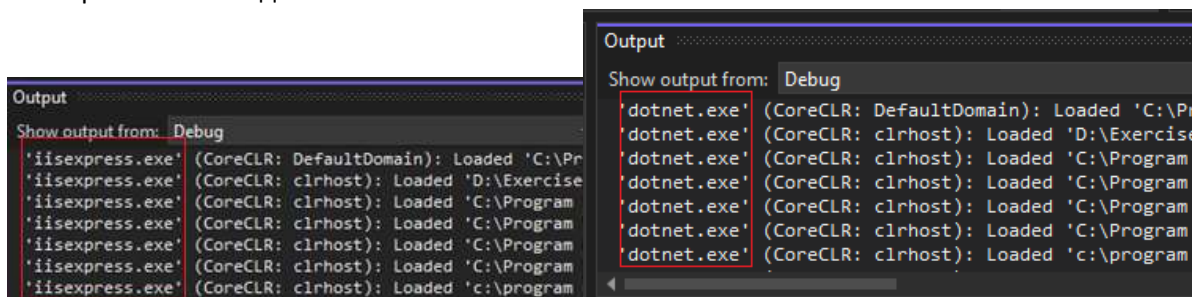


За да разберем процеса, ще стартираме в Debug mode:

И разгледаме процесите в Output window:

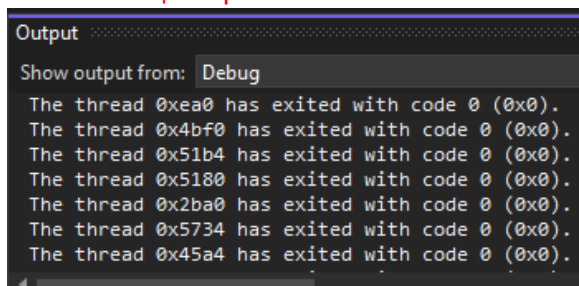


Изхвърлят се много данни:



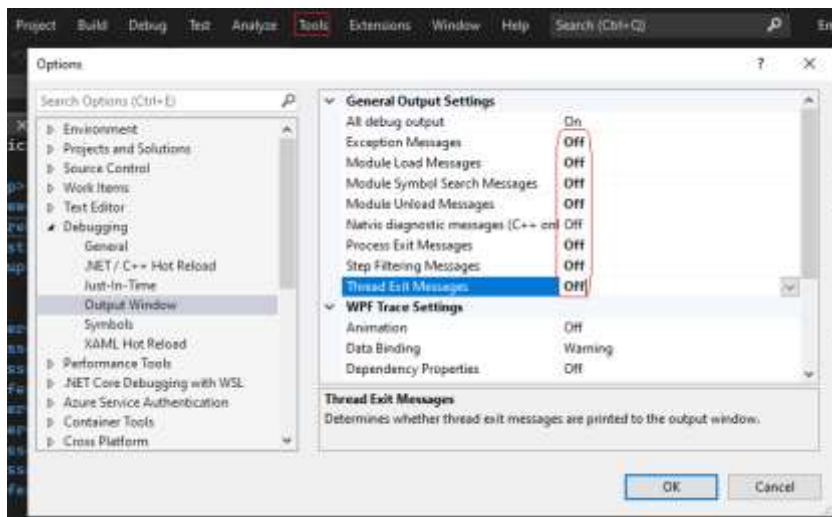
Като в първата част виждаме

iisexpress.exe, защото сме в InProcess hosting,
dotnet.exe ще е при OutOfProcess.

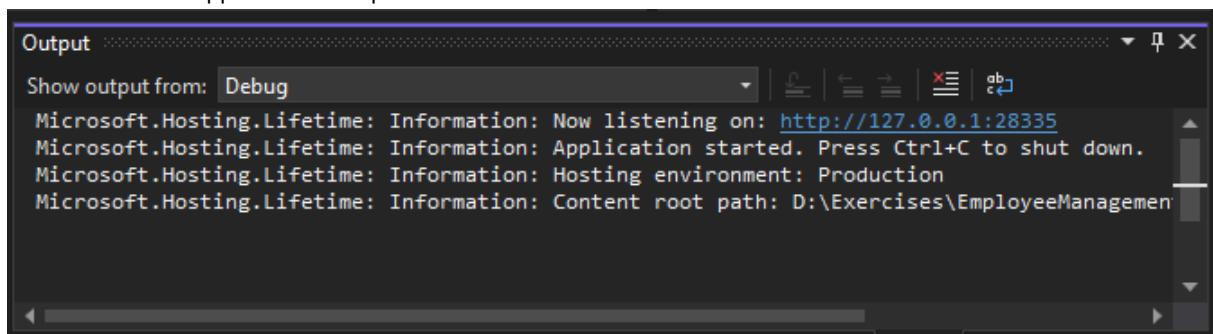


По-надолу code 0.

За да изключим всички тези излишни съобщения:

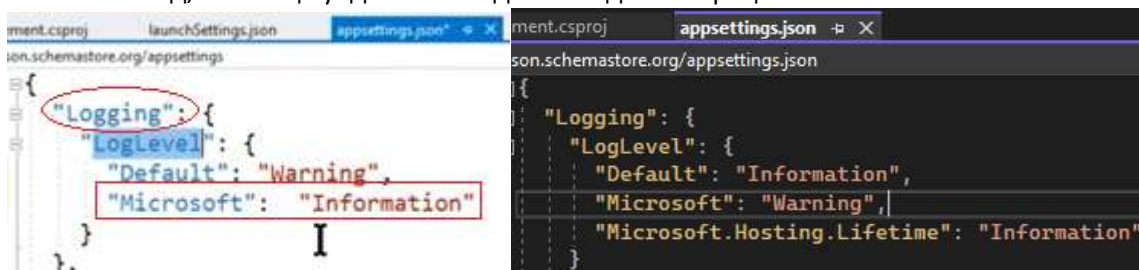


Вече нямаме подобни съобщения:

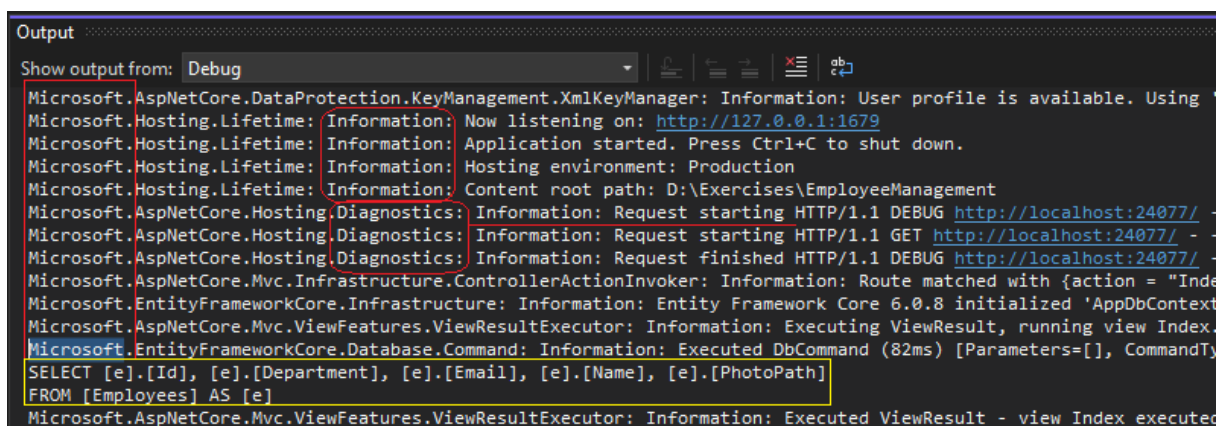


Спираме процеса.

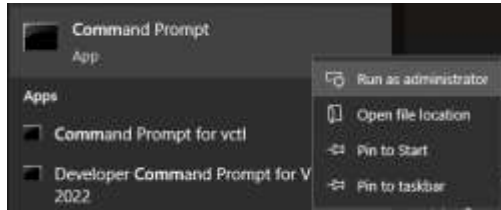
В туториъла нямаме "Microsoft": "Information" и го добавяме, а при мен по подразбиране е с повече код, който ще уеднаква с видеото за демонстрация.



По този начин имаме много информация в Debug mode:



.Net = cross-platform, можем да стартираме в Debug mode in CLI като администратор:



Select Administrator: Command Prompt - dotnet run

```
Microsoft Windows [Version 10.0.19042.631]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd /d d:\Exercises\EmployeeManagement

d:\Exercises\EmployeeManagement>dotnet run
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: d:\Exercises\EmployeeManagement
```

След зареждане на адрес Localhost:5000, всяко последващо действие се отразява в CMD, който продължава да слуша в Debug mode и извежда информация:

```
fail: Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddleware[1]
      An unhandled exception has occurred while executing the request.
      System.Exception: Error in Details View
         at EmployeeManagement.Controllers.HomeController.Details(Nullable`1 id) in d:\Exercises\EmployeeManagement\Controllers\HomeController.cs:line 29
         at lambda_method64(Closure , Object , Object[] )
         at Microsoft.AspNetCore.Mvc.Infrastructure.ActionMethodExecutor.SyncActionResultExecutor.Execute(Object mapper, ObjectMethodExecutor executor, Object controller, Object[] arguments)
         at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeActionMethodAsync()
         at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Next(State&
```

ASP.NET Core Logging Provider

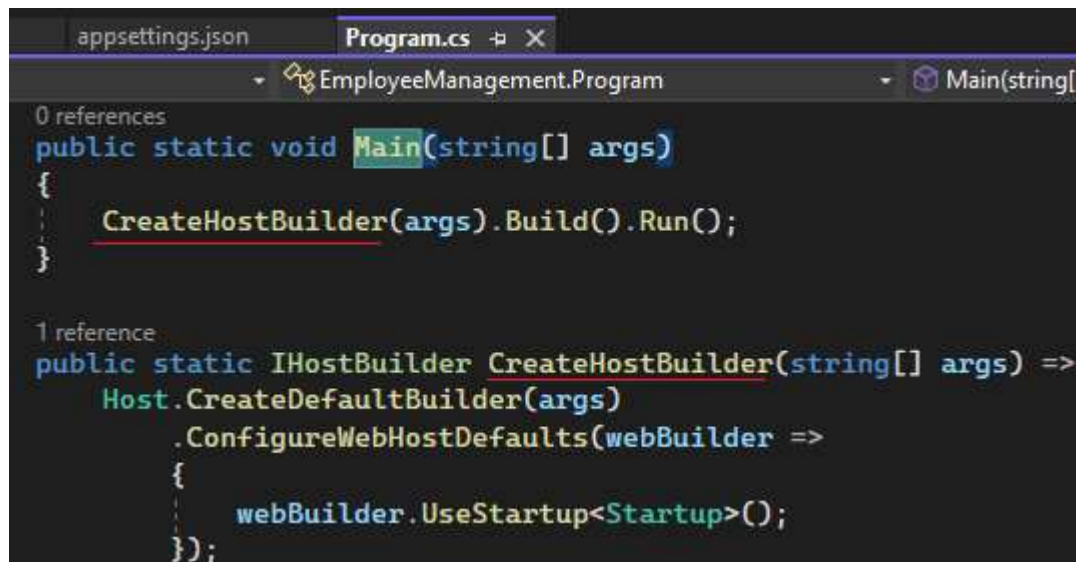
Logging Provider Stores or Displays Logs

Console Logging Provider

Displays logs on the Console

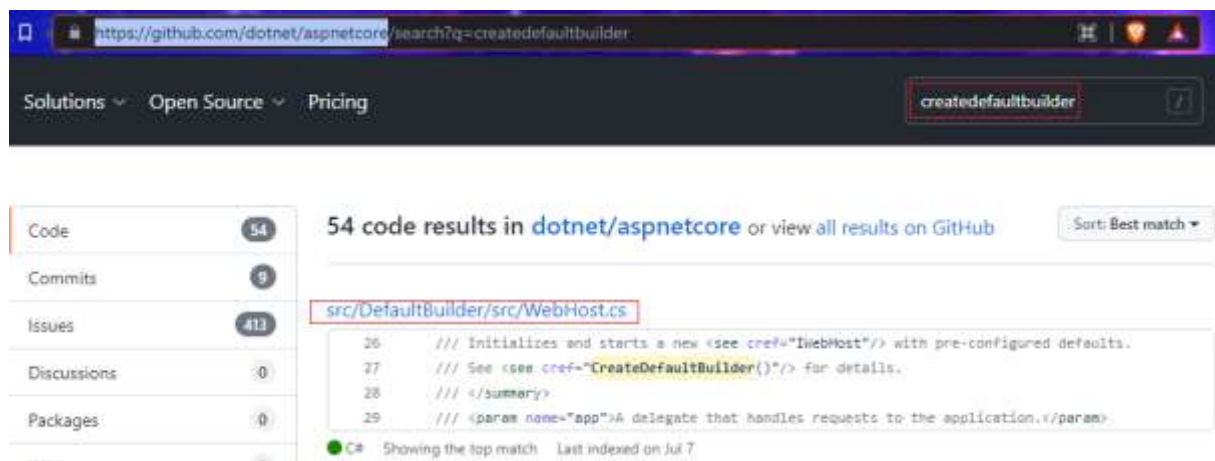
Debug Logging Provider

Displays logs on the Debug window in Visual Studio



```
appsettings.json Program.cs X
EmployeeManagement.Program Main(string[] args)
0 references
public static void Main(string[] args)
{
    CreateHostBuilder(args).Build().Run();
}

1 reference
public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
        .ConfigureWebHostDefaults(webBuilder =>
        {
            webBuilder.UseStartup<Startup>();
        })
    ;
```



https://github.com/dotnet/aspnetcore/search?q=createdefaultbuilder

Solutions Open Source Pricing createdefaultbuilder

Code 54 Commits 9 Issues 411 Discussions 0 Packages 0

54 code results in dotnet/aspnetcore or view all results on GitHub Sort: Best match

src/DefaultBuilder/src/WebHost.cs

```
26 /// Initializes and starts a new <see cref="IWebHost"/> with pre-configured defaults.
27 /// See <see cref="CreateDefaultBuilder()"/> for details.
28 /// </summary>
29 /// <param name="app">A delegate that handles requests to the application.</param>
```

Showing the top match Last indexed on Jul 7

Където виждаме, че той отговаря за Logging и се предоставят 3 възможности за целта – Console, Debug, EventSource:

```

    }

    builder.ConfigureAppConfiguration((hostingContext, config) =>
    {
        var env = hostingContext.HostingEnvironment;

        config.AddJsonFile("appsettings.json", optional: true, reloadOnChange: true)
            .AddJsonFile($"appsettings.{env.EnvironmentName}.json", optional: true, reloadOnChange: true);

        if (env.IsDevelopment())
        {
            if (!string.IsNullOrEmpty(env.ApplicationName))
            {
                var appAssembly = Assembly.Load(new AssemblyName(env.ApplicationName));
                if (appAssembly != null)
                {
                    config.AddUserSecrets(appAssembly, optional: true);
                }
            }
        }

        config.AddEnvironmentVariables();

        if (args != null)
        {
            config.AddCommandLine(args);
        }
    })
    .ConfigureLogging((hostingContext, loggingBuilder) =>
    {
        loggingBuilder.Configure(options =>
        {
            options.ActivityTrackingOptions = ActivityTrackingOptions.SpanId
                | ActivityTrackingOptions.TraceId
                | ActivityTrackingOptions.ParentId;
        });
        loggingBuilder.AddConfiguration(hostingContext.Configuration.GetSection("Logging"));
        loggingBuilder.AddConsole();
        loggingBuilder.AddDebug();
        loggingBuilder.AddEventSourceLogger();
    })
}

```

logging

<https://github.com/dotnet/aspnetcore/blob/c4efbfb1f9a3b6241140150ae9abf219bde16b2/src/DefaultBuilder/src/WebHost.cs>

Но освен тях, има още:

Built-in Logging Providers

Console

TraceSource

Debug

AzureAppServicesFile

EventSource

AzureAppServicesBlob

EventLog

ApplicationInsights

Third Party Logging Providers

NLog

Gelf

elmah

JSNLog

Serilog

KissLog.net

Sentry

Loggr